Mikrocontroller Labor 0

Inhalt

1. Vorbereitung: *LED-Board* anschließen . . . 1
2. Acht LEDs gleichzeitig ansteuern . . . 1
3. LEDs abwechselnd aufleuchten lassen . . . 1
4. Register-Wert ausgeben . . . 1
5. Programmcode auf den ATmega32 portieren . . . 1
6. *Hardware Abstraction Layer* (HAL) entwerfen . . . 2
6.1. Pin Mapping . . . 3
Links . . . 4

Mit Programmcode können am Mikrocontroller digitale Ein- und Ausgänge konfiguriert und gesteuert werden. In diesem Labor steuern Sie mit den Hardware-Pins des Mikrocontrollers acht LEDs an.

- - Eine Grundlagen-Aufgabe, die von allen gelöst werden sollte.
- - Optionale Aufgabe mit spannenden vertiefenden Inhalten.

1. Vorbereitung: *LED-Board* anschließen

Alle Ports des Mikrocontrollers sind auf dem Entwicklerboard als einzelne *Pin-Header* ausgeführt. Schließen Sie das *LED-Board* an **Port A** an.

2. Acht LEDs gleichzeitig ansteuern

Lassen Sie alle acht LEDs an **Port A** blinken. Da der Mikrocontroller weit schneller arbeitet als das menschliche Auge sehen kann, müssen die Blink-Abstände verzögert werden. Verwenden Sie dazu das `_delay_ms()` Makro (siehe [ms]).

3. LEDs abwechselnd aufleuchten lassen

Versuchen Sie über eine Bit-Operation den Programmcode aus Abschnitt 2 abzuändern, sodass abwechselnd nur die LEDs 0,2,4,6 und die LEDS 1,3,5,7 aufleuchten.

4. Register-Wert ausgeben

Geben Sie den Wert des Registers `UCSRA` mit den acht LEDs aus. Überprüfen Sie anschließend, ob der Wert mit dem *Initial Value* im Datenblatt [ad] übereinstimmt.

> Über das *LED-Board* können Sie jedes beliebige Register für *Debugging*-Zwecke ausgeben.

5. Programmcode auf den ATmega32 portieren

Folgend finden Sie Programmcode, der ursprünglich für den Raspberry Pi geschrieben wurde. Die *WiringPi-Library* wird verwendet, um den Buchstaben `C` als Morse-Code auszugeben.

Portieren Sie den Programmcode auf den ATmega32, indem Sie alle Funktionen der *WiringPi* durch direkte Registerzugriffe ersetzen. Verwenden Sie dazu die Makros aus dem *Header-File* `utils/bit.h` (von [lf]), um die Bit-Operationen zu erleichtern.

Erweitern Sie abschließend das portierte Programm, sodass es wiederholt die Initialen der Vornamen

Ihres Teams als Morse-Code ausgibt.

```
#define LED_PIN 1
#define DIT 100
#define DAH (3*DIT)
...
void ledDit()
{
  digitalWrite(LED_PIN, HIGH);
  _delay_ms(DIT);
  digitalWrite(LED_PIN, LOW);
  _delay_ms(DIT);
}

void ledDah()
{
  digitalWrite(LED_PIN, HIGH);
  _delay_ms(DAH);
  digitalWrite(LED_PIN, LOW);
  _delay_ms(DIT);
}

void morseC()
{
  ledDah();
  ledDit();
  ledDah();
  ledDit();
}
...
```

6. 🚀 *Hardware Abstraction Layer* (HAL) entwerfen

In der vorherigen Aufgabe haben Sie den Programmcode für den ATmega angepasst, und damit plattformspezifischen Programmcode geschaffen.

Um die Portabilität hingegen beizubehalten, ist es sinnvoll, einen *Hardware Abstraction Layer* (kurz: HAL) zu schaffen. Dazu ergänzen Sie die *WiringPi* Funktionen `pinMode()` und `digitalWrite()` für den ATmega. Nach dieser Aufgabe ist es somit möglich, durch einen Funktionsaufruf einen *Hardware-Pin* auf `INPUT` oder `OUTPUT` zu schalten, ohne weitere Recherchen im Datenblatt vollziehen zu müssen (siehe Abbildung 1).

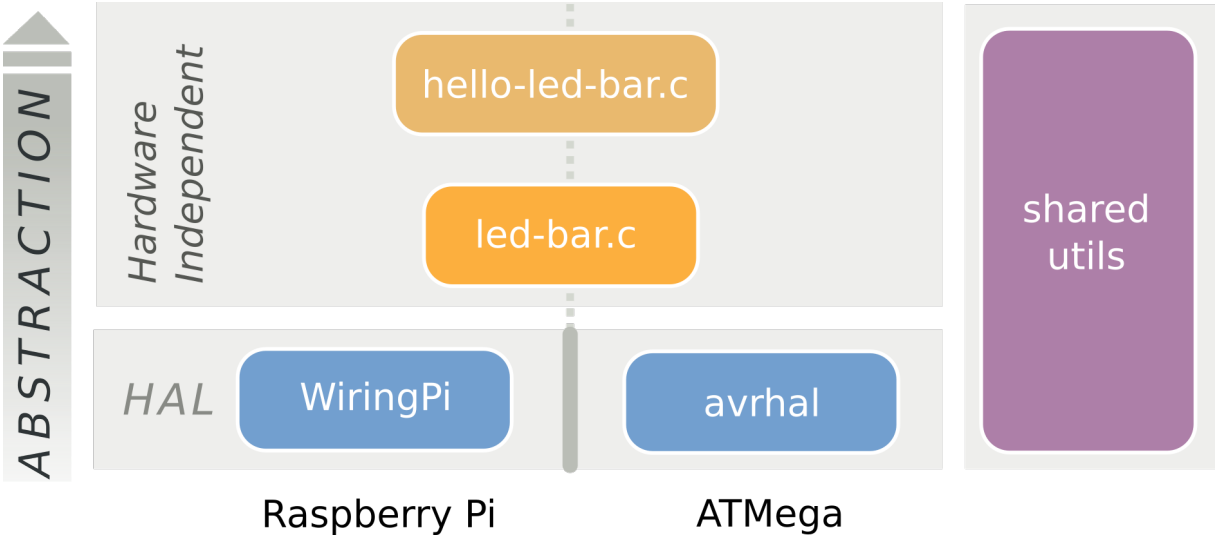

Abbildung 1. Hardware Abstraction Layer (HAL)

Ein Aufruf im *Application-Layer*, um den Pin Nr. 1 auf `OUTPUT` und `HIGH` zu setzen, sieht wie folgt aus:

```
...
pinMode(1, OUTPUT);
digitalWrite(1, HIGH);
...
```

Um Ihnen den Einstieg zu erleichtern, sind bereits *Header-Files* vorgegeben, in denen die Funktionen fertig deklariert sind (unter [lf], Ordner `avrhal/`). Der Verzeichnisbaum von Ihrem Projekt sollte somit in etwa folgendermaßen aussehen (`pin.c` und `hello-led-bar.c` müssen selbst erzeugt werden):

```
├── avrhal
│   ├── pin.h
│   └── pin.c
├── hello-led-bar.c
```

In den *Header-Files* sind bereits mehr Funktionen deklariert als Sie zunächst implementieren.

Implementieren Sie in `pin.c` die folgenden Funktionen:

- `pinMode()`
- `digitalWrite()`

Die Datei `hello-led-bar.c` erhält die `main` Funktion. Benennen Sie die Datei `main.c` um oder erstellen Sie eine neue Datei mit der `main` Funktion.

6.1. Pin Mapping

Mit dem HAL wird die hardwareabhängige Zuordnung der Pins auf Ports abstrahiert. In dem Labor wird der Einfachheit halber das folgende *Pin-Mapping* verwendet:

ATmega Port	ATMega Pin	HAL Pin
A	0 - 7	0 - 7
B	0 - 7	8 - 15
C	0 - 7	16 - 23

ATmega Port	ATMega Pin	HAL Pin
D	0 - 7	24 - 31

Das bedeutet, die Funktionen `pinMode()` und `digitalWrite()` erhalten als Eingangsparameter eine HAL Pin-Nummer von 0 bis 31 und berechnen damit intern den passenden ATmega Port sowie Pin, und greifen anschließend auf die passenden ATmega Register zu.

Realisieren Sie den Zugriff auf die I/O Register über Register-Adressen, die Sie in einem Array ablegen. Ein Beispiel:

```
static volatile uint8_t* const portDirectionRegister[] = { &DDRA, &DDRB, &DDRC, &DDRD };
```

Links

- [lf] Labor `00` Dateien - https://its-git.fh-salzburg.ac.at/mic-lb/assignment-code-ss-2025/-/tree/main/00
- [ad] ATmega32 *Datasheet* - http://ww1.microchip.com/downloads/en/devicedoc/doc2503.pdf
- [ms] https://www.nongnu.org/avr-libc/user-manual/group__util__delay.html